BEE 4750 Lab 2: Monte Carlo

Name: Pranati Patnam

ID:pp444

Due Date

Thursday, 10/02/25, 9:00pm

Setup

The following code should go at the top of most Julia scripts; it will load the local package environment and install any needed packages. You will see this often and shouldn't need to touch it.

```
import Pkg
Pkg.activate(".")
Pkg.instantiate()
Pkg.add("Extremes")
```

```
  Activating project at `c:\Users\Pranati\OneDrive - Cornell University\Documents\BEE4750-Gi
    Updating registry at `C:\Users\Pranati\.julia\registries\General.toml`
   Resolving package versions...
   Installed IntelOpenMP_jll       v2025.2.0+0
   Installed EnumX             v1.0.5
   Installed ProgressMeter         v1.11.0
   Installed FFTW              v1.10.0
   Installed StaticArrays          v1.9.15
   Installed MambaLite           v0.1.3
   Installed LazyGrids           v0.5.0
   Installed Distances           v0.10.12
   Installed ADTypes            v1.18.0
   Installed NLSolversBase         v7.10.0
```

```
 Installed FiniteDiff          v2.28.1
 Installed Loess             v0.6.4
 Installed Gadfly            v1.4.1
 Installed CommonSubexpressions     v0.3.1
 Installed oneTBB_jll          v2022.0.0+0
 Installed Compat            v4.18.1
 Installed DiffRules          v1.15.1
 Installed DiffResults         v1.1.0
 Installed PrettyTables         v3.0.11
 Installed Graphs            v1.13.1
 Installed PositiveFactorizations    v0.2.4
 Installed CategoricalArrays      v0.10.8
 Installed Setfield           v1.1.2
 Installed Hexagons           v0.2.0
 Installed MKL_jll           v2025.2.0+0
 Installed AbstractFFTs         v1.5.0
 Installed KernelDensity        v0.6.10
 Installed ArrayInterface        v7.20.0
 Installed ChainRulesCore        v1.26.0
 Installed Interpolations        v0.16.2
 Installed CoupledFields        v0.3.0
 Installed LineSearches         v7.4.0
 Installed FFTW_jll           v3.3.11+0
 Installed UnPack            v1.0.2
 Installed SimpleTraits         v0.9.5
 Installed OpenSSL_jll          v3.5.4+0
 Installed Adapt             v4.4.0
 Installed IndirectArrays        v1.0.0
 Installed Compose            v0.9.6
 Installed DifferentiationInterface  v0.7.8
 Installed Parameters          v0.12.3
 Installed ForwardDiff          v1.2.1
 Installed Optim             v1.13.2
 Installed Extremes           v1.0.5
  Updating `C:\Users\Pranati\OneDrive - Cornell University\Documents\BEE4750-GitHub\lab-2-
[fe3fe864] + Extremes v1.0.5
  Updating `C:\Users\Pranati\OneDrive - Cornell University\Documents\BEE4750-GitHub\lab-2-
[47edcb42] + ADTypes v1.18.0
[621f4979] + AbstractFFTs v1.5.0
[79e6a3ab] + Adapt v4.4.0
[ec485272] + ArnoldiMethod v0.4.0
[4fba245c] + ArrayInterface v7.20.0
[13072b0f] + AxisAlgorithms v1.1.0
```

```
 [336ed68f] + CSV v0.10.15
[324d7699] + CategoricalArrays v0.10.8
 [d360d2e6] + ChainRulesCore v1.26.0
 [bbf7d656] + CommonSubexpressions v0.3.1
 [34da2185] + Compat v4.18.1
 [a81c6b42] + Compose v0.9.6
 [187b0558] + ConstructionBase v1.6.0
 [7ad07ef1] + CoupledFields v0.3.0
 [a8cc5b0e] + Crayons v4.1.1
 [a93c6f00] + DataFrames v1.8.0
[864edb3b] ↓ DataStructures v0.19.1   v0.18.22
 [e2d170a0] + DataValueInterfaces v1.0.0
 [163ba53b] + DiffResults v1.1.0
 [b552c78f] + DiffRules v1.15.1
 [a0c0ee7d] + DifferentiationInterface v0.7.8
 [b4f34e82] + Distances v0.10.12
 [4e289a0a] + EnumX v1.0.5
 [fe3fe864] + Extremes v1.0.5
 [7a1cc6ca] + FFTW v1.10.0
 [48062228] + FilePathsBase v0.9.24
 [6a86dc24] + FiniteDiff v2.28.1
 [f6369f11] + ForwardDiff v1.2.1
 [c91e804a] + Gadfly v1.4.1
 [86223c79] + Graphs v1.13.1
 [a1b4810d] + Hexagons v0.2.0
 [9b13fd28] + IndirectArrays v1.0.0
 [d25df0c9] + Inflate v0.1.5
 [842dd82b] + InlineStrings v1.4.5
 [a98d9a8b] + Interpolations v0.16.2
 [41ab1584] + InvertedIndices v1.3.1
 [c8e1da08] + IterTools v1.10.0
 [82899510] + IteratorInterfaceExtensions v1.0.0
 [5ab0869b] + KernelDensity v0.6.10
[7031d0ef] + LazyGrids v0.5.0
 [d3d80556] + LineSearches v7.4.0
 [4345ca2d] + Loess v0.6.4
 [ee262687] + MambaLite v0.1.3
 [d41bc354] + NLSolversBase v7.10.0
 [6fe1bfb0] + OffsetArrays v1.17.0
 [429524aa] + Optim v1.13.2
 [d96e819e] + Parameters v0.12.3
 [2dfb63ee] + PooledArrays v1.4.3
 [85a6dd25] + PositiveFactorizations v0.2.4
```

```
  [08abe8d2] + PrettyTables v3.0.11
  [92933f4c] + ProgressMeter v1.11.0
  [c84ed2f1] + Ratios v0.4.5
  [91c51154] + SentinelArrays v1.4.8
  [efcf1570] + Setfield v1.1.2
  [699a6c99] + SimpleTraits v0.9.5
  [90137ffa] + StaticArrays v1.9.15
  [1e83bf80] + StaticArraysCore v1.4.3
  [892a3eda] + StringManipulation v0.4.1
  [3783bdb8] + TableTraits v1.0.1
  [bd369af6] + Tables v1.12.1
  [3a884ed6] + UnPack v1.0.2
  [ea10d353] + WeakRefStrings v1.4.2
  [efce3f68] + WoodburyMatrices v1.0.0
  [76eceee3] + WorkerUtilities v1.6.1
  [f5851436] + FFTW_jll v3.3.11+0
  [1d5cc7b8] + IntelOpenMP_jll v2025.2.0+0
  [856f044c] + MKL_jll v2025.2.0+0
  [458c3c95] ↑ OpenSSL_jll v3.5.2+0  v3.5.4+0
  [1317d2d5] + oneTBB_jll v2022.0.0+0
  [8ba89e20] + Distributed v1.11.0
  [9fa8497b] + Future v1.11.0
  [4af54fe1] + LazyArtifacts v1.11.0
  [1a1011a3] + SharedArrays v1.11.0
        Info Packages marked with  have new versions available but compatibility constraints
Precompiling project...
    950.4 ms   IndirectArrays
    885.2 ms   UnPack
   1474.9 ms   AbstractFFTs
   1197.5 ms   PositiveFactorizations
   1202.9 ms   LazyGrids
   1399.1 ms   ADTypes
   1751.7 ms   Distances
   1028.7 ms   EnumX
   1148.8 ms   Adapt
   1833.0 ms   ProgressMeter
   1058.1 ms   Compat
    729.5 ms   Hexagons
   1268.5 ms   OpenSSL_jll
   3193.6 ms   DiffResults
   3537.9 ms   CommonSubexpressions
   2948.3 ms   SimpleTraits
   1385.6 ms   oneTBB_jll
```

```
 1305.6 ms  FFTW_jll
 4308.2 ms  CategoricalArrays
 1130.1 ms  DiffRules
 2544.6 ms  IntelOpenMP_jll
 2484.3 ms  Parameters
 6792.3 ms  Setfield
 1119.4 ms  ADTypes → ADTypesConstructionBaseExt
 3385.4 ms  DifferentiationInterface
 4120.1 ms  AbstractFFTs → AbstractFFTsTestExt
  931.7 ms  ArrayInterface
 2156.2 ms  Loess
 2816.2 ms  Distances → DistancesSparseArraysExt
 2308.8 ms  Adapt → AdaptSparseArraysExt
 1993.2 ms  OffsetArrays → OffsetArraysAdaptExt
 2079.5 ms  Compat → CompatLinearAlgebraExt
 2753.4 ms  CategoricalArrays → CategoricalArraysJSONExt
 3283.0 ms  CategoricalArrays → CategoricalArraysRecipesBaseExt
 3895.3 ms  CategoricalArrays → CategoricalArraysSentinelArraysExt
10839.6 ms  StaticArrays
 2447.6 ms  DifferentiationInterface → DifferentiationInterfaceSparseArraysExt
 1631.9 ms  ArrayInterface → ArrayInterfaceStaticArraysCoreExt
 2597.4 ms  ArrayInterface → ArrayInterfaceSparseArraysExt
 1967.9 ms  StaticArrays → StaticArraysStatisticsExt
 3444.7 ms  ChainRulesCore
 4414.2 ms  DataStructures
 3285.8 ms  ArnoldiMethod
 1716.8 ms  ConstructionBase → ConstructionBaseStaticArraysExt
 1534.9 ms  Adapt → AdaptStaticArraysExt
 1155.9 ms  FiniteDiff
 1549.1 ms  DifferentiationInterface → DifferentiationInterfaceStaticArraysExt
10484.3 ms  ForwardDiff
 1141.2 ms  AbstractFFTs → AbstractFFTsChainRulesCoreExt
 3091.3 ms  ChainRulesCore → ChainRulesCoreSparseArraysExt
 1276.0 ms  ADTypes → ADTypesChainRulesCoreExt
 3656.1 ms  LogExpFunctions → LogExpFunctionsChainRulesCoreExt
 1245.8 ms  Distances → DistancesChainRulesCoreExt
 3159.9 ms  StatsFuns → StatsFunsChainRulesCoreExt
 3744.2 ms  SpecialFunctions → SpecialFunctionsChainRulesCoreExt
 1131.0 ms  ArrayInterface → ArrayInterfaceChainRulesCoreExt
 1222.9 ms  DifferentiationInterface → DifferentiationInterfaceChainRulesCoreExt
 1543.0 ms  StaticArrays → StaticArraysChainRulesCoreExt
14775.4 ms  MKL_jll
 2569.7 ms  FiniteDiff → FiniteDiffSparseArraysExt
```

```
  2162.5 ms    FiniteDiff → FiniteDiffStaticArraysExt
   976.3 ms    DifferentiationInterface → DifferentiationInterfaceFiniteDiffExt
  6217.9 ms    Compose
  2684.8 ms    DifferentiationInterface → DifferentiationInterfaceForwardDiffExt
  3491.0 ms    ForwardDiff → ForwardDiffStaticArraysExt
 12928.6 ms    CoupledFields
 13159.3 ms    Graphs
  6268.7 ms    NLSolversBase
  4120.0 ms    MambaLite
  7450.8 ms    LineSearches
 16284.3 ms    Interpolations
 16175.3 ms    FFTW
 25966.7 ms    Distributions → DistributionsChainRulesCoreExt
  4453.4 ms    Interpolations → InterpolationsForwardDiffExt
 51308.4 ms    PrettyTables
 13641.8 ms    Optim
 12018.6 ms    KernelDensity
 27325.2 ms    Gadfly
 53057.3 ms    DataFrames
  9999.8 ms    Latexify → DataFramesExt
 41646.2 ms    Extremes
 81 dependencies successfully precompiled in 153 seconds. 220 already precompiled.
 2 dependencies precompiled but different versions are currently loaded. Restart julia to a
 1 dependency had output during precompilation:
MKL_jll
  Downloading artifact: IntelOpenMP
```

```julia
using Random # random number generation
using Distributions # probability distributions and interface
using Statistics # basic statistical functions, including mean
using Plots # plotting
using Extremes
using DataFrames # Useful for handling data in a tabular format
using Distributions # Provides GEV distribution object (though Extremes.jl handles estimatio
using Gadfly # Or another plotting library like Plots.jl for visualizations
```

```
[ Info: Precompiling Extremes [fe3fe864-1b39-11e9-20b8-1f96fa57382d] (cache misses: wrong de

LoadError: ArgumentError: Package DataFrames not found in current path.
- Run `import Pkg; Pkg.add("DataFrames")` to install the DataFrames package.
ArgumentError: Package DataFrames not found in current path.
```

```
- Run `import Pkg; Pkg.add("DataFrames")` to install the DataFrames package.
Stacktrace:
 [1] macro expansion
   @ .\loading.jl:2296 [inlined]
 [2] macro expansion
   @ .\lock.jl:273 [inlined]
 [3] __require(into::Module, mod::Symbol)
   @ Base .\loading.jl:2271
 [4] #invoke_in_world#3
   @ .\essentials.jl:1089 [inlined]
 [5] invoke_in_world
   @ .\essentials.jl:1086 [inlined]
 [6] require(into::Module, mod::Symbol)
   @ Base .\loading.jl:2260
```

Overview

In this lab, we will conduct a Monte Carlo experiment and explore how sensitive Monte Carlo estimates are to the underlying probability distribution(s).

You should always start any computing with random numbers by setting a "seed," which controls the sequence of numbers which are generated (since these are not *really* random, just "pseudorandom"). In Julia, we do this with the `Random.seed!()` function.

```
Random.seed!(1)
```

```
TaskLocalRNG()
```

It doesn't matter what seed you set, though different seeds might result in slightly different values. But setting a seed means every time your notebook is run, the answer will be the same.

> **Seeds and Reproducing Solutions**
>
> If you don't re-run your code in the same order or if you re-run the same cell repeatedly, you will not get the same solution. If you're working on a specific problem, you might want to re-use `Random.seed()` near any block of code you want to re-evaluate repeatedly.

Probability Distributions and Julia

Julia provides a common interface for probability distributions with the `Distributions.jl` package. The basic workflow for sampling from a distribution is:

1. Set up the distribution. The specific syntax depends on the distribution and what parameters are required, but the general call is the similar. For a normal distribution or a uniform distribution, the syntax is

```
# you don't have to name this "normal_distribution"
#   is the mean and   is the standard deviation
normal_distribution = Normal( ,  )
# a is the upper bound and b is the lower bound; these can be set to +Inf or -Inf for ar
uniform_distribution = Uniform(a, b)
```

   There are lots of both univariate and multivariate distributions, as well as the ability to create your own, but we won't do anything too exotic here.

2. Draw samples. This uses the `rand()` command (which, when used without a distribution, just samples uniformly from the interval $[0, 1]$.) For example, to sample from our normal distribution above:

```
# draw n samples
rand(normal_distribution, n)
```

Putting this together, let's say that we wanted to simulate 100 six-sided dice rolls. We could use a Discrete Uniform distribution.

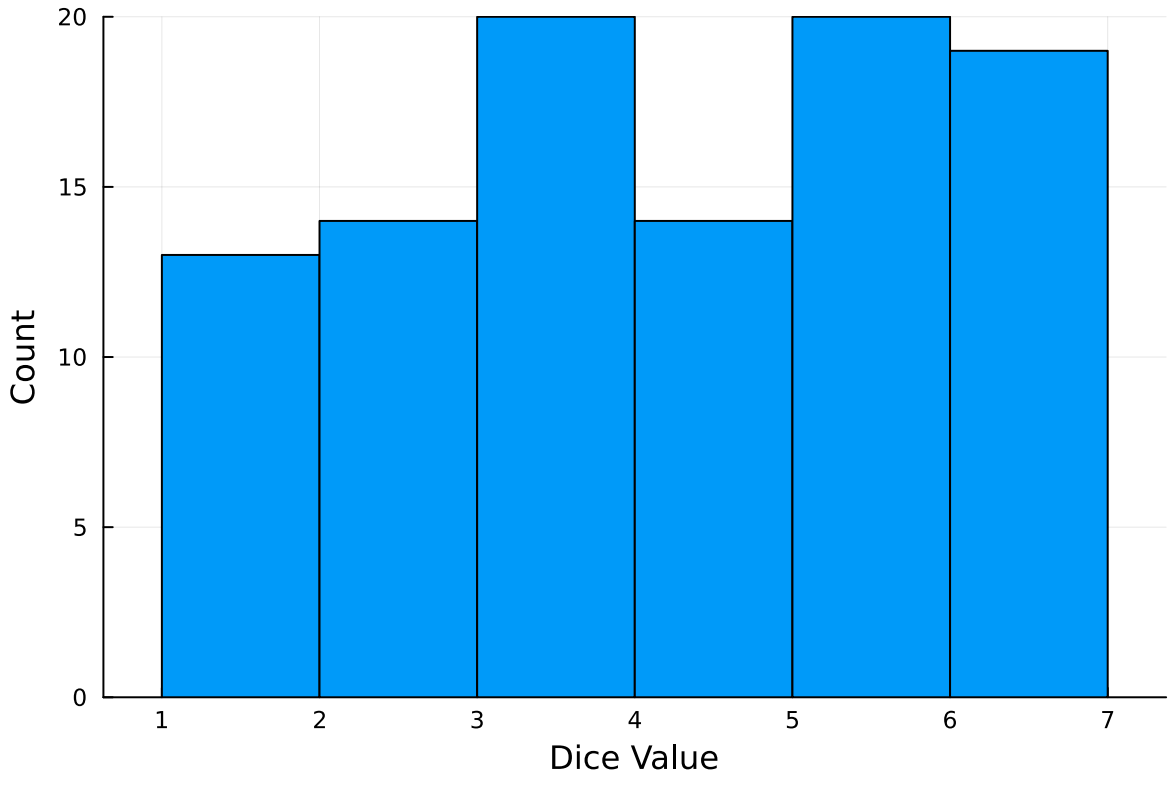
```
dice_dist = DiscreteUniform(1, 6) # can generate any integer between 1 and 6
dice_rolls = rand(dice_dist, 100) # simulate rolls
```

```
100-element Vector{Int64}:
 1
 3
 5
 4
 6
 2
 5
 5
 5
 2
 4
 3
```

```
2

2
3
3
3
6
5
5
6
3
6
6
6
```

And then we can plot a histogram of these rolls:

```
histogram(dice_rolls, legend=:false, bins=6)
ylabel!("Count")
xlabel!("Dice Value")
```



Instructions

Remember to:

- Evaluate all of your code cells, in order (using a `Run All` command). This will make sure all output is visible and that the code cells were evaluated in the correct order.
- Tag each of the problems when you submit to Gradescope; a 10% penalty will be deducted if this is not done.

Problem

A common engineering problem is to quantify flood risk, which is typically computed by propagating a flood hazard distribution through a *depth-damage function* relating flood depths to economic damages. A reasonable depth-damage function for a house without a basement is a bounded logistic function,

$$d(h) = \mathbb{1}_{h>0} \frac{L}{1 + \exp(-k(h - h_0))},$$

where d is the damage as a percent of total structure value, h is the water depth (relative to the house's ground floor elevation) in m, $\mathbb{1}_{x>0}$ is the indicator function, L is the maximum loss in USD, k is the slope of the depth-damage relationship, and h_0 is the inflection point. We'll assume $L = \$200,000$, $k = 0.8$, and $h_0 = 3$.

For this problem, suppose that we have two different probability distributions characterizing annual maxima flood depths:

1. $h \sim LogNormal(1.2, 0.3)$;
2. $h \sim GeneralizedExtremeValue(3, 0.9, -0.15)$.

Plot histograms of samples from these distributions and conduct a Monte Carlo experiment for annual maximum flood damages using each. Answer the following questions:

- What are the Monte Carlo estimates of the expected value and the 99% quantile for the annual maximum damage the structure would suffer for each of these flood hazard distributions?
- How did you decide on your sample size?
- Why do you think the estimates differed or did not differ (you can also plot the depth-damage function to compare with the samples)?
- How might you decide (based on getting additional information, if needed) which distribution is "better"?

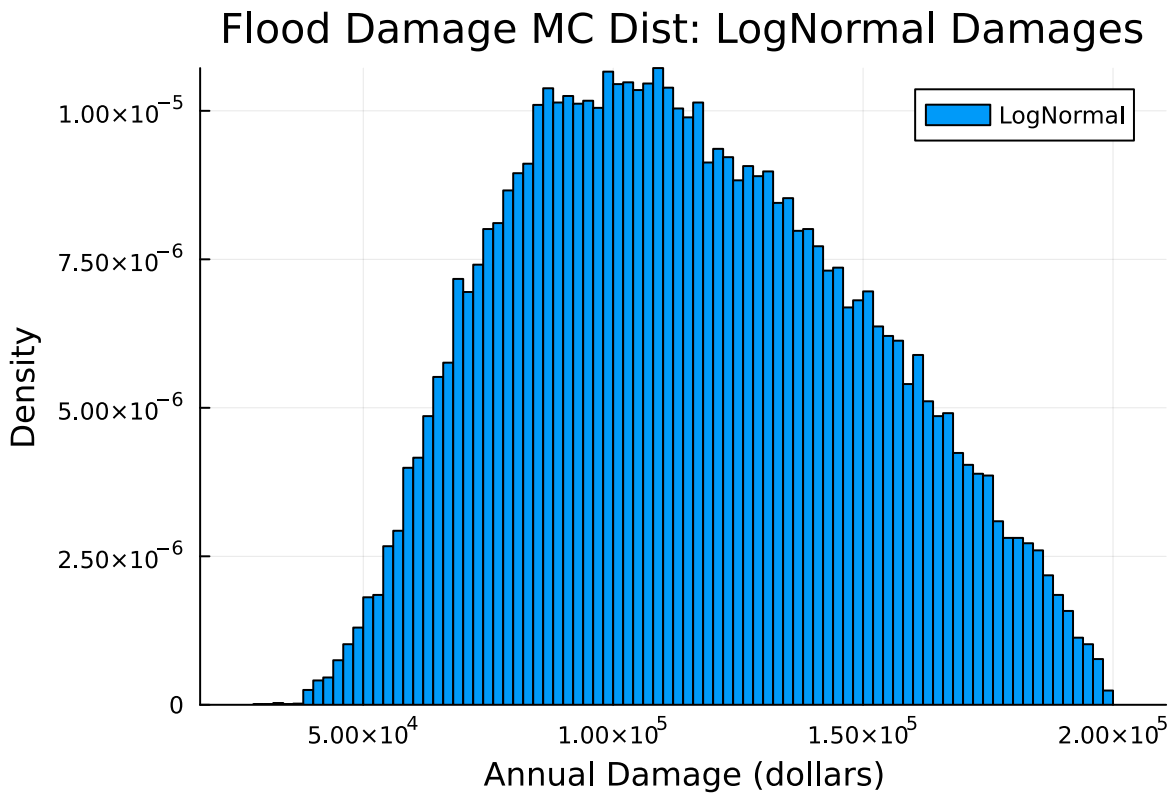
```
L  = 200_000.0
k  = 0.8
h0 = 3.0

function damage(h)
    if h <= 0 #indicator
        return 0.0
    else
        numerator = L
        expo = -k * (h - h0)
        denominator = 1 + exp(expo)
        return numerator / denominator
    end
end
```

```
damage (generic function with 3 methods)
```

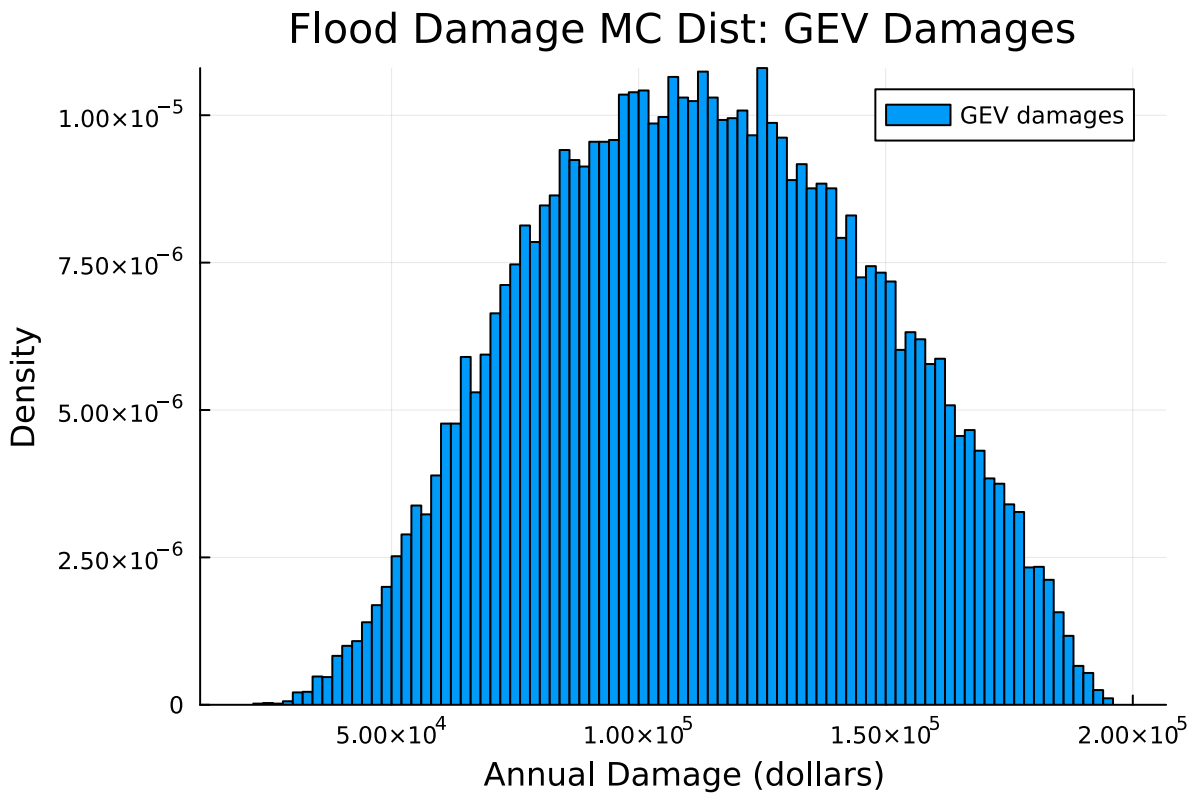
```
dist1 = LogNormal(1.2, 0.3) #lognorm
dist2 = GeneralizedExtremeValue(3.0, 0.9, -0.15) #GEV

function MC(dist; N=50000)
    hs = rand(dist, N)
    ds = damage.(hs)

    meanDam = mean(ds)
    se = std(ds) / sqrt(N)
    q99 = quantile(ds, 0.99)

    return meanDam, se, q99, ds
end
```

```
MC (generic function with 1 method)
```

```
histogram(ds1, bins=100, label = "LogNormal", normalize=true)
xlabel!("Annual Damage (dollars)")
ylabel!("Density")
title!("Flood Damage MC Dist: LogNormal Damages")
```




```
histogram(ds2, bins=100, label="GEV damages", normalize=true)
xlabel!("Annual Damage (dollars)")
ylabel!("Density")
title!("Flood Damage MC Dist: GEV Damages")
```



Question 1:

```
mean1, se1, q99_1, ds1 = MC(dist1)
mean2, se2, q99_2, ds2 = MC(dist2)

std1 = std(ds1)
std2 = std(ds2)

println("LogNormal dist:")
println("expected damage = \$$(round(mean1, digits=2))")
println("standard error = \$$(round(se1, digits=2))")
println("99% quantile = \$$(round(q99_1, digits=2))")
println("standard deviation = \$$(round(std1, digits=2))")

println()
println("GEV dist:")
println("exp damage = \$$(round(mean2, digits=2))")
println("standard error = \$$(round(se2, digits=2))")
println("99% quantile = \$$(round(q99_2, digits=2))")
println("standard deviation = \$$(round(std2, digits=2))")
```

```
LogNormal dist:
expected damage = $115179.75
standard error = $153.34
99% quantile = $190049.21
standard deviation = $34287.55

GEV dist:
exp damage = $113419.6
standard error = $150.08
99% quantile = $183575.67
standard deviation = $33558.12
```

Question 2: I ran a short pilot w/ 50000 samples to estimate the sample std dev of the damage for each distribution. If we want the standard error of the sample mean to be less, then from the slides: SE = sd/sqrt(N), and N = (sd/se)^2. Using the pilot sd gives an N 1,100–1,200 to reach SE $1,000. To be conservative and to get a more precise 99% quantile estimate, I picked N = 50000 where the actual SEs were ~$150 which is small compared to the mean (~0.1% relative SE).

for logNormal: std = 34312 N >= (34312/1000)^2 = 34.3^2 = 1177

for GEV: std = 33506 N >= (33506/1000)^2 = 33.5^2 = 1123

So roughly 1200 should be enough to be within +- 1000SE

Question 3: The means are pretty similar (115177 for LN vs 113287 for GEV). That makes sense because the depth histograms show both distributions having similar mass. the 99% quantiles due differ a little bit more than the means do (189901 for LN vs 183186 for GEV) – LN has heavier right tail compared to GEV dist, so this makes sense why the upper quantiles would be higher. I'd say both distributions kind of "compress" near the upper bound of 200k, but definitely more so with the lognormal distribution

Question 4:

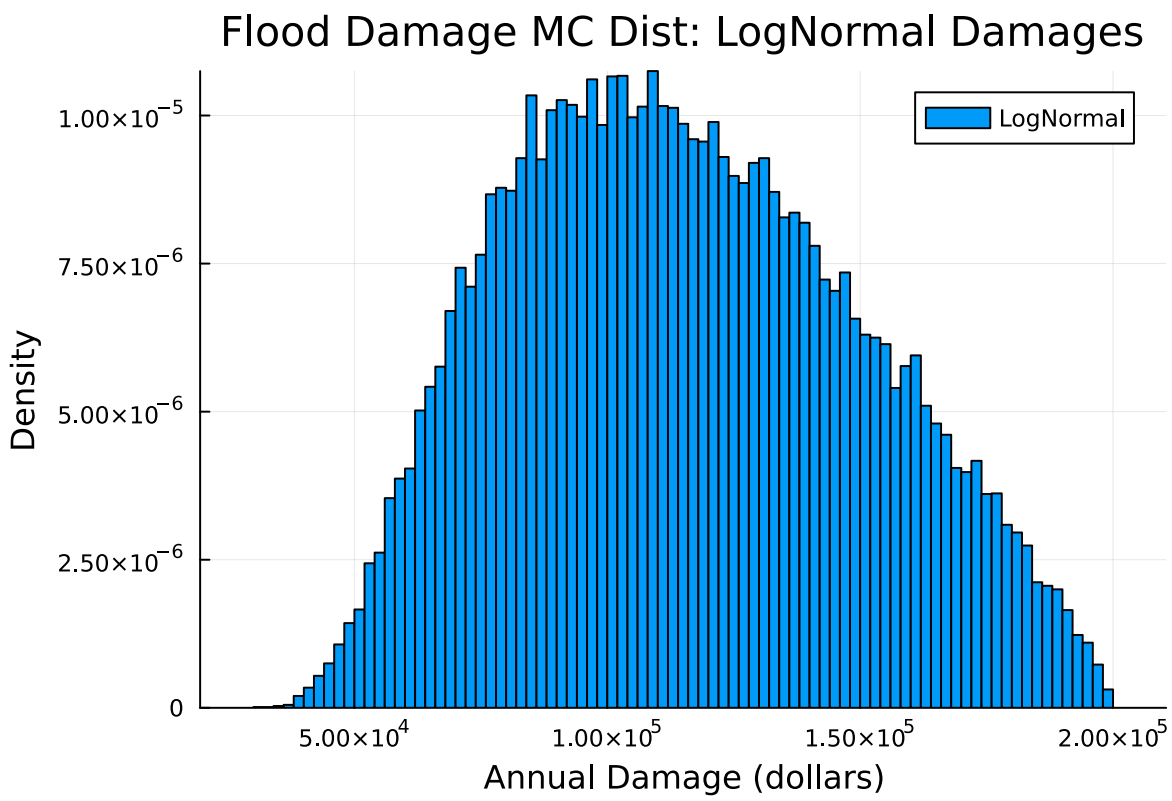
To decide which is better, I would compare tail behaviors a little better (QQ plots from 4850!), maybe include more physical constraints like other climate variables for input data (using observational data too). I could even look at monte carlo error vs model error, or run the monte carlo with different seeds to better assess the two models and their differences.

Question 5:

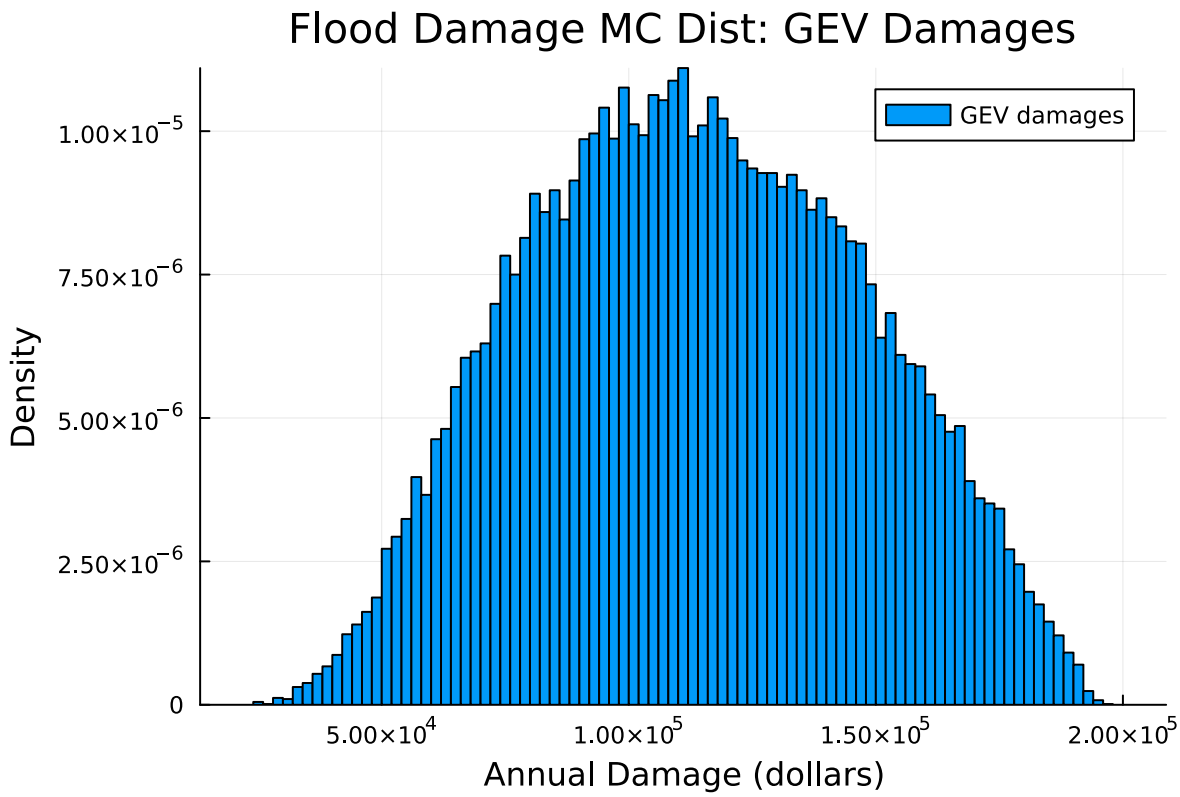
```
Random.seed!(7)
```

```
TaskLocalRNG()
```

```
mean1, se1, q99_1, ds1 = MC(dist1)
mean2, se2, q99_2, ds2 = MC(dist2)

std1 = std(ds1)
std2 = std(ds2)


histogram(ds1, bins=100, label = "LogNormal", normalize=true)
xlabel!("Annual Damage (dollars)")
ylabel!("Density")
title!("Flood Damage MC Dist: LogNormal Damages")
```




```
histogram(ds2, bins=100, label="GEV damages", normalize=true)
xlabel!("Annual Damage (dollars)")
ylabel!("Density")
title!("Flood Damage MC Dist: GEV Damages")
```




```
println("LogNormal dist:")
println("expected damage = \$$(round(mean1, digits=2))")
println("standard error = \$$(round(se1, digits=2))")
println("99% quantile = \$$(round(q99_1, digits=2))")
println("standard deviation = \$$(round(std1, digits=2))")

println()
println("GEV dist:")
println("exp damage = \$$(round(mean2, digits=2))")
println("standard error = \$$(round(se2, digits=2))")
println("99% quantile = \$$(round(q99_2, digits=2))")
println("standard deviation = \$$(round(std2, digits=2))")
```

```
LogNormal dist:
expected damage = $115221.13
standard error = $152.88
99% quantile = $189704.7
standard deviation = $34185.07
```

```
GEV dist:
exp damage = $113641.36
standard error = $150.64
99% quantile = $183367.44
standard deviation = $33683.15
```

With a random seed of 7 as the parameter instead of 1, the upper quantiles for the GEV dist are very very similar, as well as the mean. The means are similar for the logNormal dist as well, but the 99% quantile is a lot lower with this new seed. The random seed variability is small for the mean but perhaps a tiny bit more drastic for the quantiles; however this difference is not notable at all. This makes sense, since the tails would naturally be more variable.

References

Put any consulted sources here, including classmates you worked with/who helped you.

Emma P. and Adrian C-Y.